

PATHFINDER.

BY PATHFINDER LIDAR SOLUTIONS

DELIVERY

Sprint Plan

Retroactive baseline plus an eight-sprint roadmap from foundation to launch.

Project	Pathfinder Portal v2
Version	0.1 (Draft for review)
Date	2026-06-15
Owner	Pathfinder LiDAR Solutions

Contents

1. Overview	3
Methodology	3
Now / Next / Later framing	3
How sprints map to epics	4
2. Roadmap at a glance	4
3. Capacity and cadence assumptions	4
4. Sprint detail	5
S0 - Baseline (retroactive). Status: Complete (inherited)	5
S1 - Foundation. Status: Planned	6
S2 - Hierarchy. Status: Planned	7
S3 - Engine port. Status: Planned	7
S4 - Scan library. Status: Planned	8
S5 - 3D viewer. Status: Planned	9
S6 - Search and collaboration. Status: Planned	9
S7 - Compliance, accessibility, migration and launch. Status: Planned	10
5. Cross-sprint workstreams (continuous)	11
6. Milestones and suggested timeline	11
7. Risk register	12
8. Backlog and future (post-S7)	12
9. Revision history	13

Field	Value
Document	05 - Sprint Plan / Delivery Roadmap
Version	0.1
Date	2026-06-15
Status	Draft
Owner	Pathfinder LiDAR Solutions
Source of truth	docs/00-canonical-context.md (epics E1-E14, sprint map SO-S7, stack decisions)
Audience	Client stakeholders, delivery team, reviewers

1. Overview

This document is both a roadmap (where the product is going) and a sprint plan (how each increment gets built). It is deliberately a "retroactive plus forward" plan. Sprint SO does not propose new work: it documents the engines and schema already shipped in v1 (`e1s911-portal`) as the inherited baseline that later sprints port forward. Sprints S1 through S7 are the forward build of v2 (Pathfinder), a SvelteKit rewrite of the application shell that ports v1's proven imperative engines rather than rewriting them.

A roadmap is most useful when it shows progress, not just deadlines. Framing SO as Complete (inherited) and S1-S7 as Planned is the honest version of that: the foundation is real and demonstrable today, and the plan from here is sequenced to de-risk the hardest work early.

Methodology

- **Agile, two-week sprints.** Each sprint closes with a client-facing review and a team retrospective.
- **TDD-gated per epic E14.** Every acceptance criterion (AC) is proven by a failing test first (Vitest for engine and unit logic, Playwright for end-to-end flows and screenshot capture). No story is Done until its ACs pass and a Playwright screenshot is captured. Tests are written before implementation.
- **De-risking principle: shell rewrite, engine port.** The shell (auth, routing, dashboards, CRUD, state) is rewritten in SvelteKit. The imperative engines (2D annotation, map/NFPA export, 3D splat viewer) are extracted as framework-agnostic TypeScript modules mounted in thin Svelte wrappers. This is why the engine-port sprint (S3) ships early and why each engine gets its own unit tests as it is extracted.

Now / Next / Later framing

- **Now (SO-S2):** the inherited baseline plus the v2 foundation - scaffold, brand layer, auth, hierarchy. This is the standing-up phase.
- **Next (S3-S5):** the core value engines - the 2D annotation and NFPA export port, the scan library, and the new 3D splat viewer.
- **Later (S6-S7):** the cross-cutting maturity work - global search, collaboration, compliance, accessibility, data migration, and launch.

How sprints map to epics

Each sprint pulls one or more canonical epics (E1-E14) and delivers vertical slices of them. The traceability chain is Epic -> User Story (US) -> Acceptance Criterion (AC) -> Test (T) -> Sprint (S), defined in the canonical context. E14 (Quality and Delivery Harness) is not a single sprint; it is established in S1 and runs continuously through every sprint thereafter.

2. Roadmap at a glance

Sprint	Theme	Primary epics	Status	Headline deliverable
S0	Baseline (retroactive)	E5, E6 (partial), E4 (partial), E2 (partial)	Complete (inherited)	v1 2D annotation engine, map/NFPA export, projects/users/roles, D1 schema
S1	Foundation	E1, E2, E14	Planned	SvelteKit scaffold on CF, brand layer, JWT auth port, test harness, AGENTS.md
S2	Hierarchy	E3	Planned	Org -> District -> Facility -> Building model + roll-up dashboards
S3	Engine port	E5, E6	Planned	2D annotation engine + map/NFPA export ported into Svelte wrappers
S4	Scan library	E7	Planned	media_assets model + R2 multipart upload + point-cloud cold archive
S5	3D viewer	E8	Planned	mp4 walkthrough player + Spark splat viewer + 3D measurement + markers
S6	Search and collaboration	E10, E6, E9	Planned	FTS5 global search + batch map export + comment/resolve/mention/notify
S7	Compliance, ally, migration, launch	E11, E12, E13	Planned	NG911 export, immutable audit, WCAG 2.1 AA, v1->v2 migration, launch

3. Capacity and cadence assumptions

These are stated as assumptions and should be revisited at each retrospective.

- **Team size:** solo or small team (one to three engineers). Velocity figures below are illustrative, not committed; they exist to size sprints relative to each other, not to promise a date.
- **Cadence:** two-week sprints, start mid-June 2026. One client review and one retrospective per sprint.
- **TDD discipline:** tests are written before implementation. This front-loads cost in each sprint and is the reason engine-heavy sprints (S3, S5) are not also asked to carry large new surface area.
- **Continuous overhead:** roughly 15-20 percent of each sprint is reserved for the cross-sprint workstreams in Section 6 (CI, accessibility passes, security review, documentation). This is not free time; it is planned.
- **Deploy target:** every sprint deploys to `pathfinder.pages.dev` via `wrangler pages deploy` (direct push; git push does not publish). The test project is isolated from els911-portal production.

- **Definition of velocity here:** "one vertical slice" means a story whose ACs pass end to end, including UI, server route, and persistence, with a captured Playwright screenshot. Sprints are sized at three to six such slices.

4. Sprint detail

Each sprint below follows the same shape: goal, scope, task breakdown, Definition of Done, demo/acceptance, and dependencies/risks.

The Definition of Done is identical in spirit for every sprint and is stated once here, then referenced. A sprint is Done when, for every in-scope story:

- All acceptance criteria (ACs) for in-scope stories pass.
- Playwright screenshots are captured for the delivered flows.
- The build deploys clean to `pathfinder.pages.dev`.
- No secrets and no real client data are committed (the repo is public).
- The AGENTS.md rules are honored (boundaries, commands, ASCII-only commit messages, target/directories moved out before deploy).

Per-sprint sections restate the DoD with the sprint-specific additions.

SO - Baseline (retroactive). Status: Complete (inherited)

Sprint goal. Document what v1 (`e1s911-portal`) already delivers, so that the forward plan treats it as a known, tested foundation rather than greenfield.

Scope (already delivered in v1). This sprint is a record, not a plan. v1 ships, today, the following, which S3 and S6 port forward into Svelte wrappers:

- **2D PDF.js annotation engine** (`viewer.js` , ~196KB). PDF.js rendering with zoom, pan, and multi-page navigation, plus a canvas overlay supporting 12 annotation tool types: circle, rectangle, arrow, freehand, comment, correction, and the six safety markers AED, stairs, door, overhead, exit, and fire extinguisher. Full keyboard shortcut map (S/C/R/A/F/T/X/E, digits 1-6 for markers, Space to pan, arrows to page, Ctrl+Z/Y/S, Delete, +/- zoom). Undo and redo. Annotation coordinates are stored normalized to [0,1] of page dimensions, which makes them resolution-independent and portable.
- **Map markers layer.** A persistent mapping layer (`map_markers`) separate from review annotations: stairs, hallway, door, and elevator types, with pinned and manually repositioned labels, hallway polygons (JSON point arrays), and secondary label pins.
- **NFPA-style legend plus autofit.** Auto-generated standardized legend with marker placement and label autofit logic (the fiddly correctness - placement, legend autofit, jitter avoidance - that the canonical de-risking principle explicitly says not to re-litigate).
- **Multi-page print export.** Per-page crop boundaries (`map_crops`) with page labels and an explicit `print_order` for ordering multi-page printed exports.
- **Projects, users, and roles.** Projects with status (pending, in_review, approved, completed, archived) and progress (0-100); facility, district, zip, address, and phone fields used on map exports; project members with reviewer and viewer roles. Users with admin/staff/client roles. JWT auth (HS256 via Web Crypto) plus API

keys (stored as SHA-256 hashes). PBKDF2-SHA256 password hashing at 100,000 iterations. Server-side RBAC on every request; clients see only projects they are members of.

- **D1 schema.** The full v1 schema: `users`, `projects`, `project_members`, `documents`, `annotations`, `project_notes`, `annotation_comments`, `map_markers`, `map_crops`, `api_keys`, `activity_log`, `audit_log`. v2 retains all of these additively and layers the hierarchy and media tables on top (see canonical context Section 8).
- **REST API and admin panel.** Full CRUD for projects, documents, annotations, users, and API keys; JSON annotation export (schema 1.0); admin user management, API key management, permission matrix, and audit log viewer.

Task breakdown. None - work is complete. The S0 deliverable is this written baseline and the mapping of v1 capabilities to the v2 epics they feed (E5, E6, E4, E2).

Definition of Done. Met historically in v1. For the purposes of this plan, S0 is Done because the engines are demonstrably running in `els911-portal` and their behaviors are catalogued above and in the v1 README and `schema.sql`.

Demo / acceptance. Show the live v1 portal: open a floorplan, place the 12 tool types, add map markers, generate the NFPA-style legend, and produce a multi-page print export. This is the "what working looks like" reference that the forward sprints must preserve through the port.

Dependencies and risks. The principal risk that S0 introduces into the forward plan is regression during the port (addressed in S3): v1 logic is battle-tested but undertested in the unit sense, so extracting it must come with new characterization tests that lock current behavior before refactoring.

S1 - Foundation. Status: Planned

Sprint goal. Stand up the v2 SvelteKit shell on Cloudflare with the white-label brand layer, the ported auth, and the test/CI harness in place.

Scope. E1 (White-Label Brand Layer), E2 (Identity, Roles and Access), E14 (Quality and Delivery Harness). Vertical slices: a user can load the app under the Pathfinder default brand, switch to the ELS911 profile by build env, log in with JWT, and have every gate enforced server-side.

Task breakdown.

- Scaffold SvelteKit (Svelte 5) with `@sveltejs/adaptor-cloudflare`; configure Pages plus Functions; wire D1 (DB), R2 (DOCUMENTS), and KV (CACHE) bindings.
- Build the brand-profile system: `brand/<id>/brand.config.ts` plus logo, logoMark, favicon assets; flow color tokens, typography, support contacts, and legal footer to CSS custom properties at load. No hard-coded brand colors in components.
- Author two profiles: Pathfinder LiDAR (default) and ELS911 (ready-made). Build-time resolution via `BRAND=els911` env.
- Port JWT (HS256, Web Crypto) auth and API-key verification from v1 as TypeScript modules behind `+server.ts` routes; port PBKDF2-SHA256 hashing.
- Implement RBAC middleware (admin/staff/client) enforced on every server route.
- Stand up E14: Vitest config, Playwright config with screenshot capture, a thin CI workflow, and the `wrangler pages deploy` path to `pathfinder.pages.dev`.

- Author AGENTS.md (commands with flags; Always/Ask-first/Never boundaries with "never commit secrets"; flat project map with versions; test and git workflow; ASCII-only commit messages; move target/ dirs out before deploy).

Definition of Done. Standard DoD (Section 4) plus: brand swap works by env with zero component edits; login and RBAC ACs pass; AGENTS.md committed and under ~150 lines.

Demo / acceptance. Load the app as Pathfinder, redeploy as ELS911 and show the same UI reskinned by one profile swap; log in as admin/staff/client and show a route blocked for the wrong role; show a green CI run with a captured screenshot.

Dependencies and risks. No upstream dependency (this is the root sprint). Risk: brand-token plumbing that leaks a hard-coded color undermines E1 - mitigate with a lint/test that fails on literal hex in components. Risk: auth port subtly diverging from v1 - mitigate by porting the exact HS256/PBKDF2 parameters and testing against known v1 tokens.

S2 - Hierarchy. Status: Planned

Sprint goal. Add the multi-tenant org hierarchy and roll-up dashboards that become the navigational backbone for everything that follows.

Scope. E3 (Org Hierarchy and Dashboards). Vertical slices: model Org -> District -> Facility -> Building; render roll-up dashboards; provide breadcrumb switchers that double as sibling dropdowns; expose org-scoped data isolation that later auth checks rely on.

Task breakdown.

- Add `orgs`, `districts`, `facilities`, `buildings` tables; add `projects.building_id` FK; write migrations additive over the retained v1 schema.
- Build CRUD and org-scoped read paths for each hierarchy level; enforce tenant isolation in the RBAC layer from S1.
- Build roll-up dashboards (counts and status by level) with deliberate empty states (show a sample facility / preview of a completed map and one clear first action, per the UX research).
- Implement vertical sidebar primary nav (reserve top bar for few sections) and location-based breadcrumbs capped at three levels; make breadcrumb segments double as dropdown switchers to sibling nodes.
- Lay search-ready nav hooks (entity ids and labels) that S6 FTS5 will index.

Definition of Done. Standard DoD plus: hierarchy CRUD ACs pass; tenant isolation test proves a client in one org cannot read another org's tree; breadcrumb switcher and empty states captured in Playwright.

Demo / acceptance. Navigate Org -> District -> Facility -> Building, switch siblings from a breadcrumb, view a roll-up dashboard, and show a designed empty state for a brand-new facility.

Dependencies and risks. Depends on S1 (auth, bindings, brand). Risk: menu sprawl and over-nesting - mitigate with the three-level cap and search-centric nav. Risk: tenant isolation gaps - mitigate with explicit cross-tenant negative tests.

S3 - Engine port. Status: Planned

Sprint goal. Port v1's 2D annotation engine and map/NFPA export engine into framework-agnostic TypeScript modules mounted in thin Svelte wrappers, with no behavioral regressions.

Scope. E5 (2D Floorplan Annotation Engine), E6 (Safety Mapping and NFPA Export). Vertical slices: open a PDF floorplan in v2, use all 12 tools, place map markers, generate the NFPA-style legend with autofit, and produce a multi-page print export - matching v1 behavior.

Task breakdown.

- Extract `viewer.js` into a typed `annotation-engine` module: PDF.js (pinned) render, canvas overlay, the 12 tool types, normalized [0,1] coordinates, undo/ redo, keyboard map. Write characterization unit tests that lock v1 behavior before any refactor.
- Build a thin Svelte wrapper hosting the engine canvas; wire it to D1 `annotations` via `+server.ts` routes; preserve the JSON export (schema 1.0).
- Extract the map/NFPA export engine: marker placement, hallway polygons, legend autofit, multi-page print ordering (`map_markers` , `map_crops`).
- Port NFPA-style vertical legend rendering and per-page crop/print-order export to PDF; unit-test autofit and ordering against v1 outputs.
- Wire engines into the S2 hierarchy (documents under buildings/facilities).

Definition of Done. Standard DoD plus: all 12 tool ACs pass; engine unit tests (extracted per de-risking principle) pass; a v2 export visually matches the v1 reference export; Playwright captures the viewer with markers and the generated legend.

Demo / acceptance. Side-by-side with the S0 v1 reference: same floorplan, same annotations, same legend and multi-page export, now running inside the v2 Svelte shell.

Dependencies and risks. Depends on S1 (shell) and S2 (where documents live). Primary risk: 2D engine port regressions (this is the single biggest technical risk in the plan) - mitigate with characterization tests written first, a pinned PDF.js version, and the explicit rule not to re-litigate working canvas/print logic. Risk: PDF.js version drift breaking render - mitigate by pinning.

S4 - Scan library. Status: Planned

Sprint goal. Introduce the unified media model and reliable large-file upload so 3D scans, walkthroughs, and reference media have a home before the viewer is built.

Scope. E7 (Unified Scan Library and Media). Vertical slices: model `media_assets` superseding flat `documents.doc_type` ; perform R2 multipart uploads of large files; archive master point clouds to a cold tier.

Task breakdown.

- Add `media_assets` (types: `floorplan_pdf`, `point_cloud`, `splat`, `walkthrough_video`, `reference_image`) with `version` , `r2_key` , `storage_tier` (hot/cold), `capture_date` , `surveyor` . Migrate v1 `documents` semantics into it additively.
- Implement R2 multipart upload (initiate / part URLs / complete) for large files (point clouds 100-300MB, walkthrough mp4 20-100MB); resumable and size-validated client side.
- Implement the cold-archive policy: master PLY stored in a separate cold prefix and never served; delivered formats (SPZ, mp4) on the hot tier.
- Build the library UI: list and version media per building/facility with capture date and surveyor (field-verification metadata per competitive research).
- Validate uploads (type, size, before-read checks consistent with image-handling rules) and write failure-path tests.

Definition of Done. Standard DoD plus: multipart upload ACs pass including a large-file and an interrupted-upload case; storage-tier routing test proves PLY lands cold and SPZ/mp4 land hot; no real client scan data committed.

Demo / acceptance. Upload a multi-hundred-MB file via multipart, show it versioned in the library with capture date and surveyor, and show the master archived cold while the delivery asset stays hot.

Dependencies and risks. Depends on S2 (where media attaches) and S1 (R2 binding). Primary risk: large-file upload reliability - mitigate with multipart, resumability, and explicit interruption tests. Risk: storage-tier misrouting inflating egress - mitigate with a routing test.

S5 - 3D viewer. Status: Planned

Sprint goal. Ship the new browser 3D scan viewer: the zero-friction mp4 baseline plus the interactive Spark splat viewer with measurement and 3D markers.

Scope. E8 (3D Scan Viewer). Vertical slices: play the mp4 walkthrough; load an SPZ splat in Spark; measure point-to-point; place anchored 3D markers; switch floors; save named viewpoints; compare scan versions.

Task breakdown.

- Build the mp4 walkthrough player first (zero-friction baseline every responder can use; ship before the splat work).
- Integrate Spark (THREE.js, WebGL2) splat viewer; load SPZ (delivery format; master PLY never served); orbit, fly, and walk camera modes.
- Build the measurement tool (point-to-point distance, room dimensions) using Spark raycast-into-splat - a differentiator absent from off-the-shelf viewers.
- Add `scan_markers_3d` (position, normal, viewpoint, type, label, description); anchor markers in the scene; cap ~25 per scene as the sane default.
- Add `viewpoints` (named camera bookmarks: position, target, FOV) with smooth transitions; doubles as a guided key-points tour.
- Add per-floor scan switching and capture-date-labeled scan-version comparison.
- Screenshot/export for briefings.

Definition of Done. Standard DoD plus: SPZ loads and renders on a WebGL2 target; measurement ACs pass within tolerance; 3D marker persistence ACs pass; mp4 and splat viewer both captured in Playwright; performance sanity-checked on a field-representative device profile.

Demo / acceptance. Play an mp4 walkthrough, then load the same space as an interactive splat, measure a doorway, drop an AED marker with a saved viewpoint, switch floors, and compare two capture dates.

Dependencies and risks. Depends on S4 (media model and SPZ delivery). Primary risk: splat performance on field devices - mitigate by targeting WebGL2 (not WebGPU), shipping SPZ (~90 percent smaller than PLY), using Spark 2.0 streaming LOD, and never serving raw PLY (which fails above ~200MB on iPhone Safari). Risk: measurement accuracy - mitigate with tolerance-based tests.

S6 - Search and collaboration. Status: Planned

Sprint goal. Make the portal findable and collaborative: global FTS5 search, batch map export, and the anchored comment / resolve / mention / notify loop.

Scope. E10 (Global Search), E6 (Safety Mapping and NFPA Export - batch export), E9 (Collaboration). Vertical slices: search across all entity types; queue and produce a batch map PDF export; place an anchored comment that mentions a teammate and triggers a batched email with a deep link; resolve a thread without deleting it.

Task breakdown.

- Build the `search_index` FTS5 virtual table across facilities, buildings, projects, documents, and marker labels; index on write; expose a search UI wired to the S2 nav hooks.
- Add `export_jobs` (queued/tracked batch PDF map exports); build the batch export pipeline over the S3 map engine; show job status.
- Build anchored comment threads (on a map region or document area) with nested replies, building on `v1 annotation_comments`.
- Build the resolve workflow: resolve action hides (does not delete) resolved threads, preserving the audit trail.
- Add @-mentions that trigger targeted email notifications with deep links; batch notifications to avoid notification fatigue; add an activity feed and share links for outside stakeholders.

Definition of Done. Standard DoD plus: search returns ranked results across all five entity types; batch export job completes and the PDF matches the S3 single-export look; comment/resolve/mention/notify ACs pass; resolved comments are hidden not deleted (audit-trail test); notification batching verified.

Demo / acceptance. Search "Lincoln" and jump to the building; queue a batch export of every building in a district; place a pin comment, @-mention a teammate, show the batched email with a deep link, then resolve the thread and show it preserved in history.

Dependencies and risks. Depends on S2 (entities to index), S3 (map engine for batch export), and S4/S5 (media as search targets). Risk: ungoverned notification volume - mitigate with batching. Risk: FTS5 index drift - mitigate with index-on-write and a reindex path.

S7 - Compliance, accessibility, migration and launch. Status: Planned

Sprint goal. Reach procurement-grade: NG911/NENA export, immutable audit, WCAG 2.1 AA, admin/platform ops, the v1->v2 data migration, and launch.

Scope. E11 (Compliance and Trust), E12 (Accessibility), E13 (Admin and Platform Ops). Vertical slices: export NENA-aligned GIS with z-axis floor labels; immutable exportable audit logs; WCAG 2.1 AA across the app with non-visual map alternatives and a VPAT; admin tooling and observability; migrate v1 data into v2; production launch.

Task breakdown.

- Add `compliance_meta` (per-facility last-reviewed date, last responder tour, mandate flags for Alyssa's/Kari's/state, drill links, z-axis floor labels).
- Build NG911 export: GeoJSON to NENA-STA-006 schema with z-axis/floor labels per NENA-REQ-003 dispatchable location and a confidence field; build a trust page and surface compliance metadata.
- Harden audit: immutable, exportable audit logs (who viewed/edited/exported which map, when), building on `v1 audit_log` and `activity_log`.
- Accessibility pass to WCAG 2.1 AA (target 2.2 AA forward): keyboard nav, screen-reader labels on symbols and exports, contrast, and non-visual map alternatives (text room/exit lists). Produce a VPAT/ACR.
- Build admin and platform ops: user management, API keys, audit viewer, settings, and observability.

- Write and rehearse the v1->v2 data migration (users, projects, members, documents into media_assets, annotations, map markers/crops, comments, logs); dry-run against a copy; verify row counts and referential integrity.
- Production launch checklist and deploy.

Definition of Done. Standard DoD plus: NG911 export validates against the STA-006 schema with z-axis labels; audit export ACs pass and logs prove immutable; WCAG 2.1 AA automated and manual checks pass with a VPAT produced; migration dry-run reconciles row counts and FKs with zero orphans.

Demo / acceptance. Export a facility as NENA GeoJSON with floor labels; show an immutable audit export; run a screen reader through a map and read the non-visual exit list; show the admin audit viewer; demo the migration dry-run reconciliation report.

Dependencies and risks. Depends on all prior sprints. Primary risks: compliance/accessibility gaps (any one is procurement-disqualifying for gov/edu - mitigate by treating WCAG 2.1 AA and NENA conformance as launch gates) and data migration (mitigate with a dry-run against a copy, reconciliation, and a rollback plan). Risk: scope creep at the finish - mitigate by freezing scope to launch-blockers and pushing extras to the backlog (Section 9).

5. Cross-sprint workstreams (continuous)

These run through every sprint, not just S7. They are budgeted into the continuous-overhead reserve from Section 3.

- **Testing / CI (E14).** TDD throughout: failing test first, then code. Vitest for engine/unit logic; Playwright for end-to-end flows and screenshot capture. CI runs on every change; deploys go to `pathfinder.pages.dev`.
- **Accessibility (E12).** Not deferred to S7. Each sprint adds keyboard support and screen-reader labels to its new surface; S7 is the audit, VPAT, and non-visual map alternative, not the first time accessibility is considered.
- **Security / audit (E2, E11).** Server-side RBAC on every route from S1; audit logging of mutations from the start; no secrets and no real client data in the public repo, ever; periodic security review of new surface.
- **Documentation.** Living Markdown docs alongside code; ADRs for major decisions; AGENTS.md kept current; this sprint plan updated at each review to reflect actual progress (the roadmap shows progress).

6. Milestones and suggested timeline

Relative timeline only. Start is mid-June 2026; no calendar dates beyond that are invented. Each sprint is two weeks.

Milestone	Sprint(s)	Relative timing
Baseline documented (inherited)	S0	Complete before start
Foundation live on CF, brand swap demo	S1	Weeks 1-2
Hierarchy and dashboards navigable	S2	Weeks 3-4
2D engine ported, parity with v1	S3	Weeks 5-6

Milestone	Sprint(s)	Relative timing
Scan library and large-file upload	S4	Weeks 7-8
3D splat viewer with measurement	S5	Weeks 9-10
Search and collaboration loop	S6	Weeks 11-12
Compliance, ally, migration, launch	S7	Weeks 13-14

Major checkpoints: end of S3 (core 2D value re-established in v2), end of S5 (3D differentiator demonstrable), end of S7 (procurement-grade launch).

7. Risk register

Risk	Likelihood	Impact	Mitigation	Owner
2D engine port regressions	Medium	High	Characterization tests written before refactor; pin PDF.js; do not re-litigate working canvas/print logic (de-risking principle); v1 vs v2 export diff	Engine lead
Splat performance on field devices	Medium	High	Target WebGL2 not WebGPU; ship SPZ (~90 percent smaller); Spark 2.0 streaming LOD; never serve raw PLY; device-profile perf check in S5	3D lead
Large-file upload reliability	Medium	Medium	R2 multipart with resumability; size validation; explicit interrupted-upload tests; storage-tier routing test	Backend lead
Compliance / accessibility gaps	Medium	High	WCAG 2.1 AA and NENA conformance as launch gates; per-sprint ally; VPAT in S7; non-visual map alternatives; treat any single gap as procurement-disqualifying	Compliance/ally lead
Scope creep	High	Medium	Fixed two-week sprints; scope frozen to in-scope stories; extras pushed to backlog; review/retro discipline; fewer clarifying detours	Delivery lead
Data migration (v1 -> v2)	Medium	High	Dry-run against a copy; row-count and FK reconciliation; zero-orphan check; rollback plan; migration rehearsed in S7 before cutover	Backend lead
Brand-token leakage (hard-coded color)	Low	Medium	Lint/test fails on literal hex in components; tokens-only rule from E1	Frontend lead
Auth port divergence from v1	Low	High	Port exact HS256/PBKDF2 parameters; test against known v1 tokens	Backend lead

8. Backlog and future (post-S7)

Deliberately out of scope for S0-S7; sequenced after launch. Listed so reviewers can see the trajectory without it inflating the committed plan.

- **RapidSOS integration.** Push maps and dispatchable-location data into PSAPs via the industry-standard clearinghouse; absence is disqualifying for the 911 buyer, so this is the highest-priority post-launch item.

- **SSO / MFA.** Enterprise identity (SAML/OIDC SSO) and multi-factor auth; forward-looking in the canonical auth decision.
 - **Runtime per-domain brand resolution.** Map a hostname to a brand profile at runtime (the forward-looking option beyond build-time `BRAND=` resolution).
 - **Mobile responder app.** Native or PWA responder experience optimized for field use beyond the any-browser baseline.
 - **CAD / E57 export.** Paid deliverable: export scans and maps to CAD and E57 for cross-agency and facilities/insurance dual-use.
 - **SOC 2 Type II / GovRAMP.** Pursue the certifications that gov/edu procurement increasingly requires (SOC 2 broadly; GovRAMP for government).
-

9. Revision history

Version	Date	Author	Notes
0.1	2026-06-15	Pathfinder LiDAR Solutions	Initial draft; S0 documented as inherited baseline, S1-S7 planned forward build